Fyn AI & SaveTax — Architecture Map

Date: 2026-04-30 **Branch:** `feature/fyn-persona-split` **Purpose:** End-to-end map of how Fyn AI (onboarding + advice) and the SaveTax campaign are built — prompts, context, LLM calls, validation, outcomes. **Reflects:** `April30Updates` audit fixes F-1 through F-15 (all applied — see `auditFixes.md`). Net Sprint 0 invariant coverage: **34/35** ✅ (only INV-2.3.5 still deferred per spec to Sprint 1).

MAP 1 — Fyn AI: Onboarding vs Advice

1.1 The single dispatch

One if-statement decides which Fyn handles the turn. There is no orchestrator, no registry, no persona invoker — just a controller branch.

`app/Http/Controllers/Api/AiChatController.php:175–183`

```
$inOnboarding = $user->onboarding_completed === false
    && $user->onboarding_fyn_step !== null
    && (bool) config('onboarding.fyn_flow_enabled', true);

return new StreamedResponse(function () use (...) {
    $generator = $inOnboarding
        ? $this->onboardingDirector->handleUserMessage($user, $conversation, $message,
$currentRoute)
        : $this->adviceFyn->handle($user, $conversation, $message, $currentRoute);
```

Both generators yield SSE events into the same stream. The user never sees the boundary.

1.2 Onboarding Fyn — the writer

Class: `app/Services/Onboarding/OnboardingChatDirector.php` — `handleUserMessage()` at line 82.

Critical fact: *most* onboarding turns do **not** call the LLM. They are deterministic state transitions driven by `OnboardingStateMachine`. The LLM is only invoked for two turn types:

Turn type	When	LLM tools exposed
`delegated` (asset_capture / campaign_*)	User describes records in free text	Focus-filtered `create_*` + `capture_*` only
`grouped_extract` (base_personal/spouse/dependants/work)	User answers a multi-field grouped question	Narrow extraction tools (`capture_personal_details`, `capture_spouse_details`, etc.)

Bubble taps, navigation, acks, state advances, and welcome-back resumption all run with **zero** LLM calls. There is also a **parking short-circuit**: `OnboardingFactExtractor::extractAndPark` (line 102) extracts facts speculatively from every user message into `ai_conversations.onboarding_parked_facts`. If a later `grouped_extract` turn finds all needed fields already parked, it hydrates from parking and skips the LLM.

Pre-LLM duplicate guard (F-11, post-audit)

`handleAssetCaptureTurn` at line 1672 now runs `RecordDuplicateChecker::alreadyExists()` BEFORE delegating to the LLM. The focus is mapped to a checker-recognised entity_type and the user message scanned against existing DB rows; if every entity already exists, emit `"Already on file — nothing to add there."` + `done` and skip the LLM entirely. Mapping (line 1694):

onboarding_fyn_selection	Mapped entity_type	F-11 guard active?
savings, budgeting	savings_account	Yes
investment	investment_account	Yes
retirement	pension	Yes
protection	protection_policy	Yes
goals	goal	Yes
estate, business, savetax	null (fall through)	**No — handler-level dedup is the floor**

The advice path's own RecordDuplicateChecker short-circuit has been the de-dup defence for that side since S0.x; the F-11 fix mirrors it onto the onboarding side for the focuses where a single entity_type cleanly maps. SaveTax / estate / business cover multiple entity families per focus and so still rely on handler-level idempotency.

Prompt assembly — OnboardingPromptBuilder::buildAssetCapturePrompt()

Short-form (~500 tokens). Four layers, **cache-first ordering** (F-5, post-audit):

1. CoreIdentity::get($firstName) — static identity + security
2. ComplianceRules::get($taxYear) — FCA rules, hedging language, acronym policy
3. assetCaptureInstructions($focus) — the multi-entity rule with worked examples (focus-stable)
4. MemoryRetrieverService::renderKnownFactsBlock($user, $conversation) — DB + parked + conversation index, suffixed with "Do not ask the user for any field above" (INV-2.2.3, INV-2.11.1)

The reorder is the F-5 fix: pre-fix layout was CoreIdentity → ComplianceRules → known_facts → assetCapture, which invalidated Anthropic's prefix-cache from Layer 3 onward because known_facts grows after every capture. Post-fix, the first three layers (~350 tokens) are byte-stable for the duration of a focus block and benefit from cache hits. Estimated 60–70% input-token reduction on turns 2-N.

Onboarding **deliberately omits** FcaProcessInstructions, financial_context, existing_records, data_completeness, query_knowledge, kyc_result — the director owns state, so the LLM only needs the capture contract. That's why it's ~500 tokens vs ~1,800 for advice.

The multi-entity rule (the key prompt-engineering trick)

OnboardingPromptBuilder.php:139–203. The rule is hard-coded with worked examples:

- "Halifax ISA £10k and Nationwide saver £5k" → create_savings_account × 2 in **the same assistant turn**
- "Aviva life £300k and Vitality CI £100k" → create_protection_policy × 2

Plus the FR-M14 guardrail: ack ≤ 15 words, no questions, no leading text. This is purely a prompt instruction — there is **no code-side enforcement** if the LLM violates it.

Tool dispatch and writes

LLM emits tool_use blocks → CoordinatingAgent::executeTool() → handler classes (CreateSavingsAccountHandler, CreatePensionHandler, CreateFamilyMemberHandler, etc.) → direct DB writes in their own transactions → entity_created SSE event per record → appendAuditCompletion (line 965) writes the audit row AND fires AdvicePromptCacheInvalidator::forUser($user->id) on every successful write (F-9, post-audit) so the next advice turn rebuilds the prompt with fresh existing_records / financial_context.

1.3 Advice Fyn — the reader

Class: app/Services/AI/AdviceFyn.php — handle() at line 188.

Three short-circuits before the LLM

1. **Out-of-remit** (lines 223-247) — QueryClassifier::classify() returns OUT_OF_REMIT, emit canonical refusal, return. No LLM.
2. **Server-side write-intent** (line 259) — WriteIntentClassifier::classify($message) matches conservative verb+entity patterns. **F-6 (post-audit) added an interrogative guard:** if the message ends with ? OR starts with should i,

`can i`, `how do i`, `what is`, `where should`, `tell me`, `explain`, `show me` (etc., 30+ prefixes — `WriteIntentClassifier.php:100–115`), the classifier returns null and the turn falls through to the LLM. Pre-fix, "Should I add to my Cash ISA?" matched `add` + `cash isa` and bypassed the LLM straight to inline-capture, asking the user "What do you want to add?" instead of answering.

3. **Duplicate detection** (line 270) — `RecordDuplicateChecker::alreadyExists()` — if the user reasserts records that all already exist, emit a deterministic ack from DB. No LLM.

Only after all three pass does it call the LLM.

The WRITE_TOOLS strip

`AdviceFyn.php:151–175` — explicit blacklist:

```
private const WRITE_TOOLS = [
    'create_savings_account','create_investment_account','create_holding',
    'create_pension','create_property','create_mortgage',
    'create_protection_policy','create_asset','create_liability',
    'create_estate_gift','create_chattel','create_business_interest',
    'create_trust','create_family_member','create_will','update_will',
    'create_power_of_attorney','update_power_of_attorney',
    'update_record','delete_record','update_profile','set_expenditure',
    'capture_personal_details','capture_spouse_details',
    'capture_dependants','capture_work_details',
    'create_goal','create_life_event','create_what_if_scenario',
    'navigate_to_page',
];
```

`buildToolList()` at line 517-535 loads the full catalogue, adds handoff tools, then `array_diff` strips WRITE_TOOLS. Result: read-only tools + `delegate_to_capture` + `capture_complete` only. Even `navigate_to_page` is stripped (S0.5.t) to force the handoff path for any UI action.

Prompt assembly — `AdvicePromptBuilder::build()` (10+ layers)

`app/Services/AI/AdvicePromptBuilder.php:55–224`. All layers are concatenated with `\n\n`:

#	Layer	Source	Trigger
1	`CoreIdentity`	static	always
2	`ComplianceRules`	static (tax year dynamic)	always
3	`FcaProcessInstructions`	static	always
3b	`handoff_guidance`	static	`!isPreview` (promoted from layer 10b in S0.5.t)
3c	`billing_guidance`	static	classification = BILLING
3d	`known_facts` block	`MemoryRetrieverService`	non-empty
4	`<user_profile>`	DB	always
5	`<financial_context>`	sized analysis	**`engineCallLevelFor !== 'factual'` (F-14)**
6	`<existing_records>`	DB filtered by classification	**`engineCallLevelFor !== 'factual'` (F-14)**
7	data completeness	prerequisite gates	always
7b	review-due	user record	conditional
8	query knowledge	per-domain decision trees	classification-gated
8b	required tools + triggers	classification	classification-gated
9	KYC result	`KycGateChecker`	non-null
10	`<current_context>`	route	non-empty

#	Layer	Source	Trigger
11	FCA signposting	classification	ADVICE_TYPES only
12	preview mode	static	preview users

Total ~1,500–1,800 tokens for advice / module / holistic; ~700 tokens for factual after F-14 strips Layers 5/6.

F-14 (post-audit): `AdvicePromptBuilder.php:124-151` checks `AdviceFyn::engineCallLevelFor($classification['primary']) === 'factual'` and skips both `<financial_context>` (Layer 5) and `<existing_records>` (Layer 6) entirely for BILLING / NAVIGATION / DATA_ENTRY / OUT_OF_REMIT / INCOME / GENERAL queries. Pre-fix, "where's my invoice?" still carried the user's full financial position into the LLM context as ~500 tokens of irrelevant noise. Estimated 500–1000 input-token reduction per factual turn.

Sized engine calls (the cost lever)

`RESPONSE_MODE_MAP` and `ENGINE_CALL_LEVEL_MAP` at `AdviceFyn.php:52-120`. Three levels:

- `holistic` — runs all 9 module agents via `orchestrateAnalysis` (only `HOLISTIC_HEALTH` queries)
- `module` — runs only the relevant `{Module}Agent::analyze()` (e.g. ProtectionAgent for protection_cover)
- `factual` — skips engines entirely, `module_analysis = []` (income, general, data_entry, navigation, billing, out_of_remit)

`engineCallLevelFor(?string $primary)` at line 136 returns `factual` for any unknown / null primary (F-10, post-audit). Pre-fix the lenient fallback was `holistic`, so an unmapped query type ran the most expensive code path. The strict variant `engineCallLevel($queryType)` (used in tests) still throws on unmapped types so exhaustive coverage is enforced; the lenient variant defaults to the cheapest path because an unmapped primary is more likely a low-signal non-advice message than a holistic-health query.

The handoff — `wrapStream()` at line 368-474

The clever bit. The LLM emits `delegate_to_capture` as a tool call. `CoordinatingAgent` translates it into a synthetic `handoff` SSE event. `wrapStream` intercepts it and runs **two-stage validation** (F-1, post-audit):

```
if ($type === 'handoff' && $handoffType === HandoffContract::DELEGATE_TO_CAPTURE) {
    $payload = (array) ($event['payload'] ?? []);

    // 1. Hard validation — entity_types missing or wrong type
    $validationError = HandoffPayloadValidator::validateDelegateToCapture($payload);
    if ($validationError !== null && $validationError !== 'missing_or_invalid_reason') {
        Log::warning(...);
        yield ['type' => 'handoff_error', 'reason' => $validationError, 'message' => "I
couldn't pick up that request — could you try again?"];
        yield ['type' => 'done'];
        return;
    }

    // 2. Soft validation — only `reason` missing → log notice, recover via synthesis
    if ($validationError === 'missing_or_invalid_reason') {
        Log::notice('[AdviceFyn] delegate_to_capture payload missing reason — recovering via
CaptureContext synthesis', ...);
    }

    $context = CaptureContext::fromArray($payload);  // synthesises reason from entity_types
when missing
    yield from $this->onboardingChatDirector->handleInlineCapture(
        $user, $conversation, $message, $context, $currentRoute,
    );
    return;  // S0.5.t — terminate Advice generator
}
yield $event;
```

`HandoffPayloadValidator` (`app/Services/AI/HandoffPayloadValidator.php`) returns one of:

- `null` — payload valid
- `'missing_or_invalid_reason'` — soft, recover via `CaptureContext::fromArray` synthesis
- `'missing_or_invalid_entity_types'` — hard, emit `handoff_error`
- `'entity_types_must_be_strings'` — hard, emit `handoff_error`

Frontend handling (`resources/js/store/modules/aiChat.js:542–557`): a `handoff_error` SSE event commits a normal assistant content bubble with the error message ("I couldn't pick up that request — could you try again?") and logs at warn level. Per INV-2.4.3 there is **no special chrome** — the user sees an ordinary Fyn message.

Three invariants enforced by this block:

1. `handoff` event is **never yielded** to the frontend (INV-2.4.1) — stripped
2. No `persona_state_change` event ever exists
3. The `return` after handoff terminates Advice Fyn so the LLM doesn't echo "I've recorded X" after onboarding already emitted `entity_created`

Inline capture

`OnboardingChatDirector::handleInlineCapture()` is the same writer used during onboarding, but called with `persistUserMessage: false` (Advice already saved the message), `personaOverride: 'data_capture'`, and a wider tool whitelist (`captureToolSet()`, includes `update_*` and `delete_*`).

1.4 Onboarding vs Advice — side-by-side

Aspect	Onboarding Fyn	Advice Fyn
Entry	`OnboardingChatDirector::handleUserMessage()`	`AdviceFyn::handle()`
Triggers LLM?	Only on `delegated` / `grouped_extract` turns	Once per turn (after 3 short-circuits)
Prompt tokens	~500	~1,500–1,800 (module/holistic) / ~700 (factual after F-14)
Prompt layers	4 (Core / Compliance / AssetCapture / KnownFacts — F-5 cache-first order)	12+ (incl. financial_context + existing_records skipped on factual per F-14)
Tools exposed	Focus-filtered `create_*` + `capture_*` (~8 per focus)	All read tools + handoff (~40+), WRITE_TOOLS stripped
Writes?	Yes (sole writer)	No — only via `delegate_to_capture` handoff
Persona tag	`data_capture`	`advice`
Pre-LLM dedup guard	`RecordDuplicateChecker` for savings/investment/retirement/protection/goals (F-11); SaveTax / estate / business fall through	`RecordDuplicateChecker` for every classified write intent
Prompt cache	**Anthropic prefix cache hits Layers 1-3** (Core + Compliance + assetCapture) — known_facts moved last in F-5 so the prefix is byte-stable across the 6-8 SaveTax / multi-turn flow	Anthropic `cache_control: ephemeral` on system prompt; `cacheReadInputTokens` captured for telemetry (F-4)
Tool-call cap	Constant `MAX_TOOL_CALLS_PER_TURN = 5` (default, no engine signal)	**Dynamic — holistic 8 / module 5 / factual 3 (F-15)**, keyed off `AdviceFyn::engineCallLevelFor`
Transient retry	Inherits `HasAiChat` retry — single 1.5s backoff on 429/529/timeout when no partial output (F-13)	Same
Model / temp / max	Default model, T=0.7, 4096 tokens (8192 pro)	Same

Aspect	Onboarding Fyn	Advice Fyn
State	Driven by `OnboardingStateMachine`	Stateless

1.5 Prompt-engineering patterns

- **Reusable static fragments** in `app/Services/AI/Prompts/`: `CoreIdentity`, `ComplianceRules`, `FcaProcessInstructions` — single source for both Fyns. F-7 deleted the dead `getDataCreationGuidance()` method that contradicted `<handoff_guidance>` and survived as a re-enablement risk.
- **Classification-gated layers** — billing_guidance, query knowledge, FCA signposting, required tools — only injected when the classification calls for them. F-14 extends this to Layers 5/6 (financial_context + existing_records skipped on `factual`).
- **Memory layer** — `MemoryRetrieverService::renderKnownFactsBlock()` queries DB + parked facts + conversation index across 4 sources to prevent re-asking. Cited fix: INV-2.11.1.
- **Sized engine output** — `analyzeRelevantModules` only runs the agents the classification needs. Eval-driven fix flagged on 2026-04-28. F-10 hardens the unknown-primary fallback to `factual` (was `holistic`).
- **Anthropic prompt caching** — system prompt only, `cache_control: ephemeral`. Tools and message history are not cached. **F-4** adds capture of `cacheReadInputTokens` (and snake-case fallback) on `RawMessageStartEvent` so `ai_messages.metadata.cache_hit_rate` is visible for Anthropic, not just xAI. Onboarding now benefits from prefix caching too after the F-5 reorder put `known_facts` last.
- **Tool-result compression (F-3)** — `HasAiChat::compressToolResultForModel(string $toolName, array $result): array` (line 891) trims tool results before they're re-injected into the LLM message history. Errors pass through verbatim (the model must surface them per `<available_actions>` WRITE-failure rule); handoff/navigate/fill_form actions pass through; direct-write results trim to `{success, entity_type, entity_id, name}`; read tools recursively trim list arrays >10 items to head + `__truncated__: N items omitted` + tail, depth ≥3 collapses to `[nested N items]`, strings >200 chars truncated. Estimated ~60% reduction in input tokens on holistic chats with multiple tool calls.
- **Few-shot** — only in `assetCaptureInstructions` and `handoff_guidance`. Worked examples train the multi-entity emit pattern and the immediate-handoff pattern.
- **No full RAG** — knowledge is structured static blocks, not vector retrieval.
- **No JSON mode / no schema enforcement** — the LLM emits native tool_use blocks; SDK handles schema. Failed tool calls log but don't crash.
- **Prompt-injection mitigation** — `UserContentSanitiser::wrap($firstNameRaw)` wraps user-controlled strings in `<user_provided>` markers; CoreIdentity has explicit anti-injection language. **F-2** (post-audit) replaced the original whitelist (`[A-Za-z0-9\s'.,\-]`) with a denylist (`[<>{}\[\]();"|\\$@#&=+*^~/:?]+ control chars +\p{Cf}\p{Co}\p{Cn}\p{So}`). The whitelist locked out non-ASCII names ("François" → "Franois", "　" → ""), creating an inclusivity bug AND a memory-consistency bug — the LLM saw a different name than the DB stored. The denylist preserves Unicode letters while still blocking every character used in known prompt-injection vectors.
- **Token budgets** — daily caps in `HasAiGuardrails`: pro 500k, student 100k, preview 50k. History truncation at 20 messages (`HasAiChat::MAX_HISTORY_MESSAGES`).
- **Dynamic tool-call cap (F-15)** — `HasAiChat::TOOL_CALL_CAPS_BY_LEVEL = ['holistic' => 8, 'module' => 5, 'factual' => 3]` (line 59). Every turn reads the cap once via `AdviceFyn::engineCallLevelFor($classification['primary'])`. Pre-fix the constant was 5 — holistic chats genuinely needed 5+ tools (orchestrate + 3 module analyses + 1 calculation), so capping at 5 truncated reasoning chains and forced a tools-disabled retry. Default fallback (`MAX_TOOL_CALLS_PER_TURN = 5`) still applies when the engine level is unrecognised (e.g. onboarding asset_capture).
- **Single transient retry (F-13)** — `HasAiChat` catch-block classifies the exception via `isRetriableLlmError(\Throwable $e): bool` (line 1110). On retriable error AND no partial output yet (`$toolCallCount === 0 && $fullResponse === ''`), `usleep(1_500_000)` and `continue` the loop. `$turnRetried` flag prevents re-retry within the same turn. Retry on: 429 / 529 / `rate_limit` / `overloaded` / `capacity` / `timeout` / `connection` / `service_unavailable` / `temporarily`. Do NOT retry on: auth / `api_key` / 401 / 403 / `invalid_request` / `context_length` / `max_tokens` / `tool_use_id`. Mid-turn failures after tool calls have run are **not** retried — that would re-execute completed tool work.
- **System prompt persistence (F-8)** — assistant messages now write `'system_prompt' => 'sha256:'.hash('sha256', $systemPrompt)` (line 711) instead of the 1500–1800 token full prompt. The full prompt embeds user PII (income, family names, financial position) and was ~10KB per assistant message; storing it on every row of a 100-message conversation = ~1MB of metadata bloat per conversation plus a redundant copy of data already canonical in `users` / `family_members` / module tables. The hash is enough to confirm prompt-structure changes when debugging. The `sha256:` prefix lets a future reader distinguish hashes from legacy full-prompt rows. The DB column is still named `system_prompt` (rename requires a separate migration).

- **Cache invalidation on capture (F-9)** — `app/Services/AI/AdvicePromptCacheInvalidator.php::forUser(int $userId): void` forgets `ai_existing_records_{userId}` and every `ai_financial_context_{userId}_{primary}` variant (iterated via `(new ReflectionClass(QuerySchemas::class))->getConstants()` plus `unknown`). Hooked from `CoordinatingAgent::appendAuditCompletion` (line 987) on every audited tool call where `operationFor($toolName) === 'write'` and no error. Pre-fix, a user creating a record via inline-capture and immediately asking an advice question saw stale `existing_records` (60s TTL) and `financial_context` (120s TTL) — the LLM might say "you don't have an ISA" or attempt a duplicate create. Cache forget failures are caught and logged so they never bring down the write path.
- **Outcome validation** — minimal. State-machine gating + FR-M14 ack guardrail (prompt-only). FR-M14 (ack ≤ 15 words) is prompt-only — no code-side enforcement. Tool handlers return errors but no AI-driven recovery.

Eval bypass — defence-in-depth (F-12)

The `bypass-preview-mode` Sanctum ability (issued by `EvalAuthController::login` in non-production) lets eval write tools through preview-mode filtering and write interception. Pre-fix the ability alone was sufficient — a leaked or misconfigured token = preview-data corruption risk. **F-12** introduces `app/Services/Eval/EvalBypassGate.php::isActive(?User $user): bool` which now requires BOTH:

1. The active Sanctum token has the `bypass-preview-mode` ability, AND
2. The request carries a non-empty `X-Eval-Run-Id` header.

Wired at three checkpoints:

- `HasAiChat.php:166-167` — preview-mode tool filter check
- `CoordinatingAgent::executeTool` — same gate before dispatching write tools in preview mode
- `Http/Middleware/PreviewWriteInterceptor.php:142-147` — middleware-level write block

The eval harness (`EvalHttpDriver::run`) mints one run-id per `evaluate()` call (`'eval-' . bin2hex(random_bytes(8))`) and adds the header to every authenticated request (create conversation, send message, logout). A stolen token alone is no longer sufficient; the access pattern is also more visible in logs (the run-id surfaces in nginx access logs and is easy to alert on). Server-side allowlist of in-flight run-ids is a follow-up.

MAP 2 — SaveTax campaign

2.1 The headline finding

SaveTax's strategy generation is fully deterministic. The LLM is used only during onboarding for data extraction. Once data is captured, `TaxStrategyCalculator` does pure rule-based math against `TaxConfigService`. There is no AI-generated language anywhere in the strategy output.

That is a clean and defensible architecture, but it's worth knowing — there is no "LLM picks strategies" loop to tune.

2.2 Trigger chain

Step	File	Notes
Landing	`resources/js/views/Public/SaveTaxCampaignPage.vue:1-220`	Public marketing page; CTA → `/register?from=savetax`
Register	`resources/js/views/Register.vue`	Carries `from=savetax` query param
Dashboard intercept	`resources/js/views/Dashboard.vue`	Reads `$route.query.from` before router strips it, dispatches `aiChat/startOnboardingConversation`
API entry	`app/Http/Controllers/Api/AiChatController.php:265-410` (`POST /api/ai-chat/onboarding/start`)	Reads `from`, looks up `config('onboarding.campaign_map')`
State init	Same controller, lines 341-376	Sets `users.onboarding_fyn_path='campaign'`, `selection='savetax'`, `step='base_personal'`

Step	File	Notes
Config	config/onboarding.php:75–77	'campaign_map' => ['savetax' => 'savetax'] — adding a campaign is a config change

2.3 The state machine path

app/Services/Onboarding/OnboardingStateMachine.php. SaveTax follows the standard base flow then branches at STATE_PROFILE_REVIEW_EXPENDITURE:

```
BASE_PERSONAL → BASE_SPOUSE → BASE_DEPENDANTS → BASE_EMPLOYMENT
  → BASE_WORK (LLM grouped_extract) → BASE_EXPENDITURE
  → PROFILE_REVIEW_EXPENDITURE
  → CAMPAIGN_INTRO (consent gate "okay" / "nope")
    → CAMPAIGN_OCCUPATIONAL_SCHEME (LLM)
    → CAMPAIGN_ISA_HOLDINGS (LLM)
    → CAMPAIGN_BANK_ACCOUNTS (LLM)
    → CAMPAIGN_INVESTMENT_ACCOUNTS (LLM)
    → CAMPAIGN_PENSION_CONTRIBS (LLM)
    → CAMPAIGN_SPOUSE_WORK (LLM, sets household_calculation_mode)
       → if dual_earner: CAMPAIGN_SPOUSE_HOUSEHOLD (LLM)
       → if single_earner_couple: CAMPAIGN_SPOUSE_NON_WORKING_ASSETS (LLM)
    → CAMPAIGN_TERMINAL (celebration + navigate /tax-strategy)
```

Each ```` BASE_PERSONAL → BASE_SPOUSE → BASE_DEPENDANTS → BASE_EMPLOYMENT → BASE_WORK (LLM grouped_extract) → BASE_EXPENDITURE → PROFILE_REVIEW_EXPENDITURE → CAMPAIGN_INTRO (consent gate "okay" / "nope") → CAMPAIGN_OCCUPATIONAL_SCHEME (LLM) → CAMPAIGN_ISA_HOLDINGS (LLM) → CAMPAIGN_BANK_ACCOUNTS (LLM) → CAMPAIGN_INVESTMENT_ACCOUNTS (LLM) → CAMPAIGN_PENSION_CONTRIBS (LLM) → CAMPAIGN_SPOUSE_WORK (LLM, sets household_calculation_mode) → if dual_earner: CAMPAIGN_SPOUSE_HOUSEHOLD (LLM) → if single_earner_couple: CAMPAIGN_SPOUSE_NON_WORKING_ASSETS (LLM) → CAMPAIGN_TERMINAL (celebration + navigate /tax-strategy)

```
Each `CAMPAIGN_*` state is a `delegated` turn — same handler as the journey-flow
`asset_capture`, just with a different focus tag (`savetax`) and a different tool list.

## 2.4 Pre-LLM context

Captured deterministically before the campaign branch:
- `users.first_name`, `date_of_birth`, `marital_status` (BASE_PERSONAL)
- Spouse/dependants → `family_members` rows
- `users.employment_status`, `employer`, `occupation`, `annual_employment_income` (BASE_WORK —
this is one of the few base states that uses the LLM via `grouped_extract`)
- `users.expenditure_monthly` (BASE_EXPENDITURE)

This is the data that gets injected into the system prompt as `known_facts` for every campaign
LLM call, so the model doesn't re-ask.

## 2.5 Prompt construction for SaveTax LLM turns

**`OnboardingPromptBuilder::buildAssetCapturePrompt()`** — same builder as the journey flow.
Four layers in **cache-first order** (F-5):

1. `CoreIdentity::get($firstName)` — name wrapped in `<user_provided>` markers, sanitised via
the F-2 denylist
2. `ComplianceRules::get($taxYear)` — current tax year only, **no allowance amounts injected**
3. `assetCaptureInstructions('savetax')` — multi-entity rule, FR-M14 guardrail, **the focus-
specific tool list**
4. `MemoryRetrieverService::renderKnownFactsBlock()` — what we already know (income, DOB,
marital, prior captures); placed LAST so the cacheable prefix above stays byte-identical across
the 6–8 turns
```

```
### Tool list for `focus='savetax'`

`OnboardingPromptBuilder.php:118-127`:
```

create_pension capture_salary_sacrifice create_savings_account create_investment_account create_holding capture_spouse_work_status capture_spouse_household_data capture_spouse_non_working_assets

```
Plus ````
create_pension
capture_salary_sacrifice
create_savings_account
create_investment_account
create_holding
capture_spouse_work_status
capture_spouse_household_data
capture_spouse_non_working_assets
```

Plus `update_profile` and `update_record` for corrections (line 131).

What is NOT in the prompt

- No allowance amounts (£20k ISA, £60k pension AA, etc.) — the model never sees current limits
- No tax computations
- No `financial_context` / `existing_records` blocks (those are advice-only)
- No few-shot of the savetax-specific scenarios — it relies on the generic multi-entity examples (Halifax ISA / Vanguard etc.)

Prompt caching

Now active (post-F-5). Anthropic's prefix cache hits on Layers 1-3 (CoreIdentity + ComplianceRules + assetCaptureInstructions) for turns 2-N of the SaveTax flow. Only the trailing `known_facts` block changes between turns, so the static prefix (~350 tokens) is reused. Estimated 60–70% input-token reduction on turns 2-N. The hit rate is now visible via `cacheReadInputTokens` capture (F-4) on `ai_messages.metadata.cache_hit_rate`.

2.6 LLM call

Site: `OnboardingChatDirector::handleAssetCaptureTurn()` line 1672.

```php
yield from $this->coordinatingAgent->chatWithPromptOverride(
    user: $user, conversation: $conversation, message: $message,
    currentRoute: $currentRoute,
    systemPromptOverride: $prompt,           // the 4-layer build
    allowedTools: $toolsForFocus,            // 8 tools above + update_profile + update_record
    persistUserMessage: false,
    toolsListOverride: null,
    personaOverride: null,
);
```

SDK: Anthropic Laravel SDK via ````php yield from $this->coordinatingAgent->chatWithPromptOverride(user: $user, conversation: $conversation, message: $message, currentRoute: $currentRoute, systemPromptOverride: $prompt, // the 4-layer build allowedTools: $toolsForFocus, // 8 tools above + update_profile + update_record persistUserMessage: false, toolsListOverride: null, personaOverride: null,);

```
**SDK:** Anthropic Laravel SDK via `HasAiChat` trait.
**Streaming:** yes (SSE).
**Temperature/max_tokens:** defaults from `HasAiGuardrails` (T=0.7, 4096/8192).
**Thinking mode:** off.
**Prompt cache:** prefix cache fires on the static three-layer head (F-5). Telemetry:
`cacheReadInputTokens` captured per turn (F-4).
```

Retry: single 1.5s backoff on 429 / 529 / network timeout when `$toolCallCount === 0 && $fullResponse === ''` (F-13).
Tool-call cap: `MAX_TOOL_CALLS_PER_TURN = 5` (default — onboarding has no engine-level signal so the dynamic F-15 cap falls back to the constant).
Tool result re-injection: every tool result is compressed via `compressToolResultForModel` before being added to the messages array (F-3).

2.7 What gets checked

Pre-LLM
- Eligibility gates in the state machine (`skipIfNotMarried`, `skipIfNotEmployed`, etc.) — skip campaign states that don't apply
- Consent gate at `CAMPAIGN_INTRO` — explicit "okay" required to proceed
- Household-mode routing (`nextFromSpouseWork`) — routes to dual_earner vs single_earner_couple branch based on the `capture_spouse_work_status` LLM output
- **F-11 duplicate-check guard does NOT activate for SaveTax.** `selection='savetax'` falls through to `default => null` in the focus-to-entity_type match at `OnboardingChatDirector.php:1694` because the SaveTax campaign covers multiple entity families per turn (savings_account, investment_account, pension, holding, capture_*). Handler-level idempotency remains the floor for SaveTax. The F-11 guard fires for the journey-flow focuses (savings, investment, retirement, protection, goals).

Tool-handler validation (`CoordinatingAgent` write handlers)
- Required-fields presence
- User ownership of records being modified
- Field allow-listing (`array_intersect_key` for spouse household data)
- Returns `{error, error_type, ...}` on failure — **no auto-retry**, error surfaces to frontend, user re-enters manually. (Note: F-13 retries network/rate-limit failures of the LLM call itself, not domain validation errors from tool handlers.)
- After every successful write, `appendAuditCompletion` fires `AdvicePromptCacheInvalidator::forUser` (F-9) so a follow-up advice query sees the new state.

Not checked at capture time
- Whether ISA contribution exceeds £20k
- Whether pension contribution exceeds annual allowance
- Whether values are plausible (someone could enter £100k pension contribution)
- These get **displayed** in the dashboard grid post-onboarding; the user is trusted to spot inconsistencies

LLM-side guardrails (prompt-only, not enforced)
- FR-M14: ack ≤ 15 words, no questions
- Multi-entity rule: emit one tool_use per record
- Compliance: "Any reference to figures the user did not provide in this message is a compliance breach"

2.8 Strategy generation — `TaxStrategyCalculator` (no LLM)

`app/Services/Tax/TaxStrategyCalculator.php:1-430`. Pure deterministic. Reads:

```php
$income = $this->taxConfig->getIncomeTax();         // PA, basic-rate band, higher-rate
$isa = $this->taxConfig->getISAAllowances();        // £20k
$pension = $this->taxConfig->getPensionAllowances(); // £60k AA, MPAA, etc.
$cgt = $this->taxConfig->getCapitalGainsTax();
$div = $this->taxConfig->getDividendTax();
```

Plus from DB: ````php $income = $this->taxConfig->getIncomeTax(); // PA, basic-rate band, higher-rate $isa = $this->taxConfig->getISAAllowances(); // £20k $pension = $this->taxConfig->getPensionAllowances(); // £60k AA, MPAA, etc. $cgt = $this->taxConfig->getCapitalGainsTax(); $div = $this->taxConfig->getDividendTax();

Plus from DB: `users` (income, marital, household_calculation_mode), `dc_pension`, `savings_account`, `investment_account`, `holding`, `tax_strategy_household_inputs`.

Generates an 8-position allowance grid for the user (and spouse if applicable) and a list of **rule-based** suggestions:
- Marriage Allowance transfer (if `marriage_allowance_eligible`)
- Savings → spouse asset shifting (if `single_earner_couple`)
- ISA top-up (if spouse has unused allowance)
- GIA to spouse (if user has investments and spouse has tax capacity)
- GIA rebalancing / ISA coordination (dual_earner mode)

Output: `TaxStrategyOutputDTO`. **No language generation.** The frontend renders the structured fields with hard-coded copy.

2.9 Delivery

Endpoint	Purpose
`GET /api/tax-strategy`	Initial dashboard — calculator runs once, returns DTO
`POST /api/tax-strategy/calculate`	Slider recalc — overrides applied in-memory, no DB write

Frontend: `TaxStrategyDashboard.vue` renders `AllowanceGrid`, `HouseholdView`, `StrategySliderPanel`, `StrategyRecommendationList`. All deterministic Vue rendering of DTO fields.

2.10 SaveTax data shape

New tables/columns added for SaveTax:
- `tax_strategy_household_inputs` — spouse income, ISA, PSA band, unrealised gains, dividends, pension input (working spouse) and existing balances (non-working spouse). Migration `2026_05_03_000003`.
- `users.marriage_allowance_eligible`, `users.household_calculation_mode` — migration `2026_05_03_000001`.

Existing tables reused: `users`, `dc_pension`, `savings_account`, `investment_account`, `holding`, `family_members`, `ai_conversations`, `ai_messages`, `onboarding_progress`.

2.11 Linear narrative — what happens for one user

1. User clicks Start trial on `/savetax` → registers with `?from=savetax`.
2. Dashboard auto-opens Fyn, dispatches `POST /api/ai-chat/onboarding/start {from: 'savetax'}`.
3. Controller sets path/selection/step, creates conversation.
4. State machine drives `BASE_*` states: deterministic bubbles + one LLM call (`BASE_WORK` grouped_extract for employer/role/income).
5. At `CAMPAIGN_INTRO`, consent gate. User taps "Okay".
6. **6–8 LLM calls** through `CAMPAIGN_*` states. Each builds the 4-layer prompt (cache-first per F-5), exposes 1–3 capture tools, writes directly to DB on tool_use. Each successful write triggers `AdvicePromptCacheInvalidator::forUser` (F-9). Tool results compressed before re-injection (F-3).
7. `CAMPAIGN_TERMINAL` emits navigation event → `/tax-strategy`. `users.onboarding_completed=true`.
8. Frontend mounts `TaxStrategyDashboard` → `GET /api/tax-strategy`.
9. **No LLM here.** `TaxStrategyCalculator` reads DB + `TaxConfigService`, returns DTO.
10. User drags sliders → `POST /api/tax-strategy/calculate` → calculator re-runs with overrides → updated DTO. Still no LLM.

Summary observations

Architectural strengths
- The two-Fyn split is genuinely simple — one if-statement, two classes, no orchestrator. Easy to reason about.
- Tool-level write isolation (`WRITE_TOOLS` blacklist + `captureToolSet` whitelist) means Advice Fyn cannot write even if prompt-injected.
- The `wrapStream` handoff is the only piece of cleverness, and it's well-bounded. F-1 closed the silent-handoff-failure gap (INV-2.4.5) — malformed payloads now surface a `handoff_error` SSE event instead of being swallowed.
- SaveTax's deterministic strategy layer avoids the entire class of LLM hallucination on tax

```
numbers.
- The April30 audit-fix bundle (F-1 to F-15) tightens the floor without adding architectural
surface: contract enforcement (F-1), inclusivity (F-2), cost (F-3, F-5, F-14), telemetry (F-4),
correctness (F-6, F-10, F-11), housekeeping (F-7, F-8), freshness (F-9), security (F-12),
reliability (F-13, F-15). Net Sprint 0 invariant coverage moved from 33/35 to 34/35; only INV-
2.3.5 (`advice_response` SSE event) remains, deferred per spec to Sprint 1.

**Things worth knowing**
- SaveTax now benefits from prompt caching across the 6–8 turn flow — the F-5 reorder put
`known_facts` last so the static prefix is byte-stable. ~60–70% input-token reduction on turns
2–N. Hit rate is observable via `ai_messages.metadata.cache_hit_rate` on Anthropic too (F-4).
- A single retry on transient LLM failures (429 / 529 / timeout) masks most rate-limit and
overload events from the user, BUT only when `$toolCallCount === 0 && $fullResponse === ''` —
mid-turn failures after tool calls have run still surface as errors because replaying would re-
execute completed tool work (F-13).
- FR-M14 (ack ≤ 15 words) is prompt-only. If long acks start appearing in production, the only
lever is prompt tuning, not code enforcement.
- `existing_records` and `financial_context` deliberately don't appear in onboarding prompts.
F-11 closes the duplicate-create risk for journey-flow focuses (savings / investment /
retirement / protection / goals) by running `RecordDuplicateChecker` BEFORE the LLM call — but
SaveTax / estate / business focuses fall through and still rely on handler-level idempotency.
- SaveTax accepts whatever values the user enters at capture time — no sanity bounds. The
dashboard simply displays them. Worth flagging if "ISA balance £200k" type entries appear.
- The `bypass-preview-mode` Sanctum ability is now defence-in-depth: requires the `X-Eval-Run-
Id` header AND the ability (F-12). A leaked or accidentally-broadcast token alone cannot bypass
preview filtering.
- Tool results are now compressed before being sent back to the LLM (F-3). This is invisible at
the API boundary but materially changes the input-token profile of holistic chats —
get_module_analysis no longer pays its 5–10KB cost on every subsequent loop iteration in the
same turn.
- System-prompt persistence is now a `sha256:` hash, not the full text (F-8). Old `ai_messages`
rows retain the full prompt; legacy data can be backfilled in a separate migration. Debugging
that requires the actual prompt text needs to regenerate it from the user state at the time,
using the hash to confirm the structure matches.
```